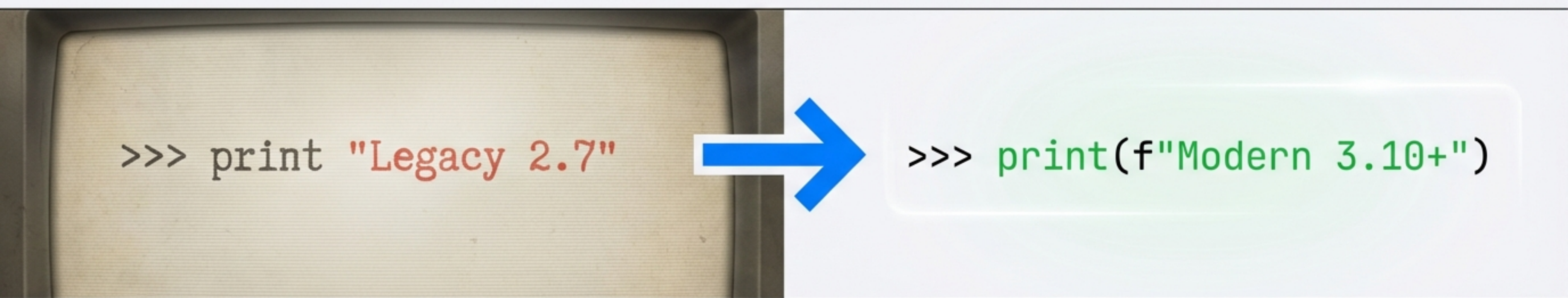


PYTHON A TRAVÉS DEL TIEEMPO

Evolución y Modernización

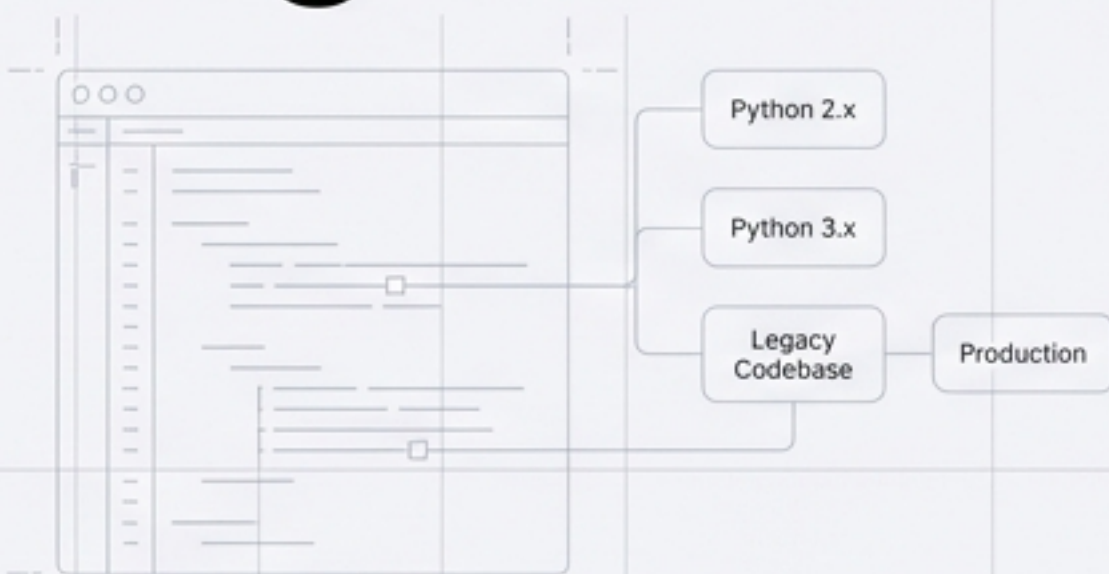


```
>>> print "Legacy 2.7"
```

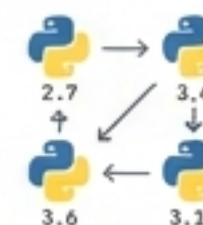
```
>>> print(f"Modern 3.10+")
```

Objetivos de la Sesión Técnica: Identificar diferencias críticas, analizar riesgos de ejecución y aplicar estrategias de migración a Python 3.10+.

El Desafío Profesional: La Realidad del Código Legado



El Contexto



En el desarrollo profesional es habitual encontrarse con proyectos creados en distintas versiones de Python.

El Riesgo

No conocer las diferencias no es una opción. La ignorancia técnica puede provocar:

- ❗ • Errores de sintaxis en tiempo de ejecución.
- ❗ • Incompatibilidades de librerías.
- ❗ • Fallos críticos en producción.

Análisis Forense I: Marcadores de Sintaxis

Fragmento A (Legacy / Python 2)

```
print "Hola mundo"
```

Identificador Python 2. El uso de **`print`** como sentencia (sin paréntesis) es exclusivo de versiones antiguas. Error de sintaxis en Python 3.

Fragmento D (Modern / Python 3.6+)

```
nombre = "Ana"  
print(f"Hola {nombre}")
```

Identificador Python 3.6+. Introducción de las **`f-strings`** para **interpolación de cadenas**. Sintaxis moderna y optimizada.

Análisis Forense II: Lógica y Control de Flujo

Matemáticas (División)



```
print(5 / 2)
```

En Python 2, esto devuelve 2 (división entera por defecto).
En Python 3, devuelve 2.5 (división real). Un cambio silencioso pero crítico.

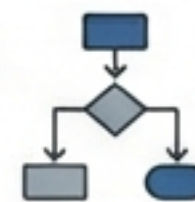
Entrada de Datos (Input)



```
numero = input(...)
```

Comportamiento ambiguo. En Python 2 evalúa la entrada (riesgo de seguridad). En Python 3 devuelve siempre una cadena (string).

Control de Flujo (Match)



```
match opcion:  
    case 1:
```

Característica exclusiva de Python 3.10+. Introduce el Structural Pattern Matching, inexistente en versiones previas.

Base de Conocimiento: I/O y Sintaxis Fundamental

Característica	Python 2 (Legacy)	Python 3 (Moderno)
Sintaxis de Print	Sentencia: print "Texto" <pre>print "Hola mundo"</pre>	Función: print("Texto") . Requiere paréntesis. <pre>print("Hola mundo")</pre>
Funcionamiento de Input	raw_input() devuelve string; input() evalúa código (peligroso). <pre>nombre = raw_input("...") valor = input("...") ⚠</pre>	input() reemplaza a raw_input() y devuelve siempre string. <pre>entrada = input("...")</pre>

Justificación: La unificación de input en Python 3 elimina vectores de ataque **por inyección de código** presentes en la función **input()** original de Python 2.

Base de Conocimiento: El Motor Interno

Característica	Python 2 (Legacy)	Python 3 (Moderno)
División (/)	Entera si los operandos son enteros.	Siempre división real (float). // para entera.
Gestión de Texto	ASCII por defecto. Manejo complejo.	UTF-8 por defecto. Soporte nativo universal.
Estado de Soporte	End of Life (2020). Obsoleto.	Activo y en constante evolución.
Nuevas Características	N/A	F-strings (3.6+), Walrus := (3.8+), Match (3.10+).

Auditoría del Entorno de Ejecución

```
$ python --version
Python 3.10.4
```

Checklist de Verificación

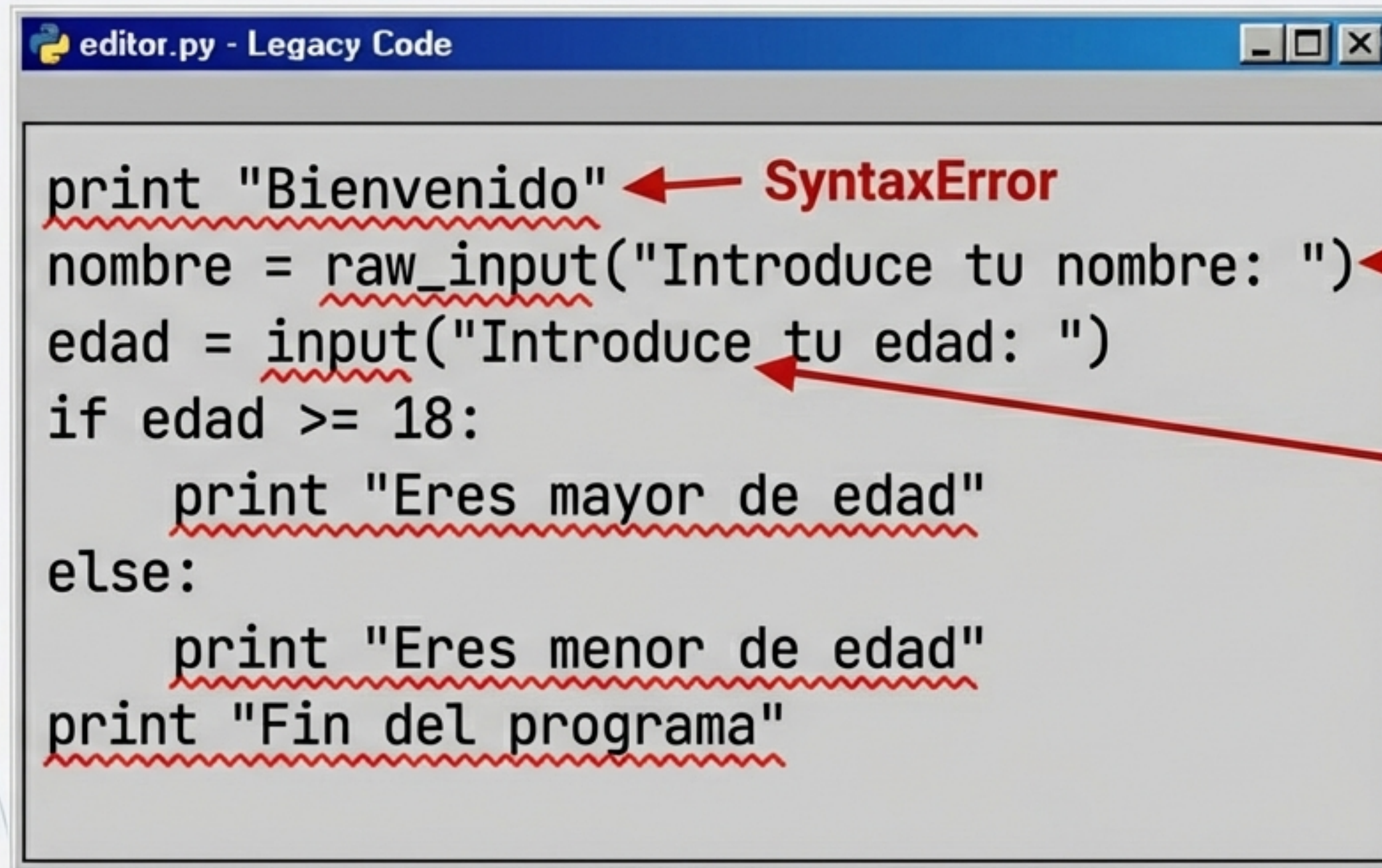
- ✓ **Versión Instalada:** Identificar si es < 3.0 (Legacy) o > 3.0 (Moderno).
- ✓ **Soporte de F-Strings:** Solo disponible si la versión es ≥ 3.6 .
- ✓ **Soporte de Structural Matching:** Solo disponible si la versión es ≥ 3.10 .



¿Qué ocurre si ejecutamos código Python 2 aquí?
SyntaxError inmediato en sentencias **print** sin paréntesis.

Caso de Estudio: El Código Legado

Variante Avanzada - Fragmento Original



```
editor.py - Legacy Code

print "Bienvenido"
nombre = raw_input("Introduce tu nombre: ")
edad = input("Introduce tu edad: ")
if edad >= 18:
    print "Eres mayor de edad"
else:
    print "Eres menor de edad"
print "Fin del programa"
```

The image shows a code editor window with the title 'editor.py - Legacy Code'. The code inside is a simple Python script. Red wavy underlines are placed under the strings 'Bienvenido', 'Fin del programa', and the two print statements in the conditional block. Red arrows point from external text labels to specific parts of the code: one to the first print statement, one to the 'raw_input' function, and one to the 'input' function.

SyntaxError

**NameError
(raw_input)**

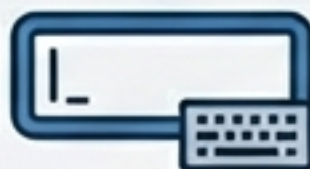
**Riesgo de
Seguridad**

Estrategia de Modernización



Estandarización de I/O

Reemplazar sentencias `print` por llamadas a función `print()`.
JetBrains Mono



Migración de Entrada de Datos

Sustituir `raw_input` por `input`.



Sanitización y Tipado

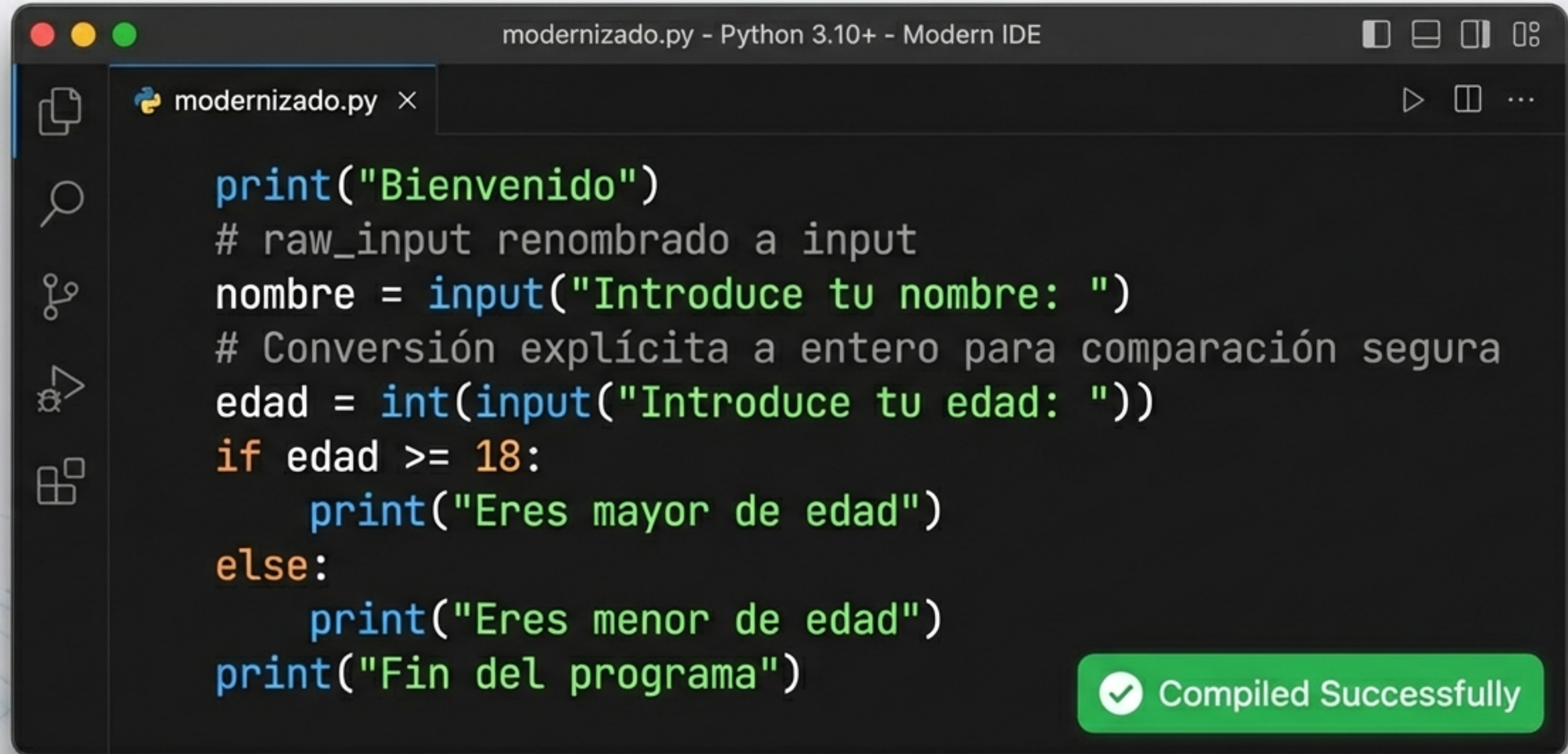
Convertir explícitamente la edad a entero usando `int()` para permitir la comparación lógica.



Validación de Flujo

Asegurar compatibilidad con Python 3.10+.

Ejecución: Código Modernizado (Python 3.10+)



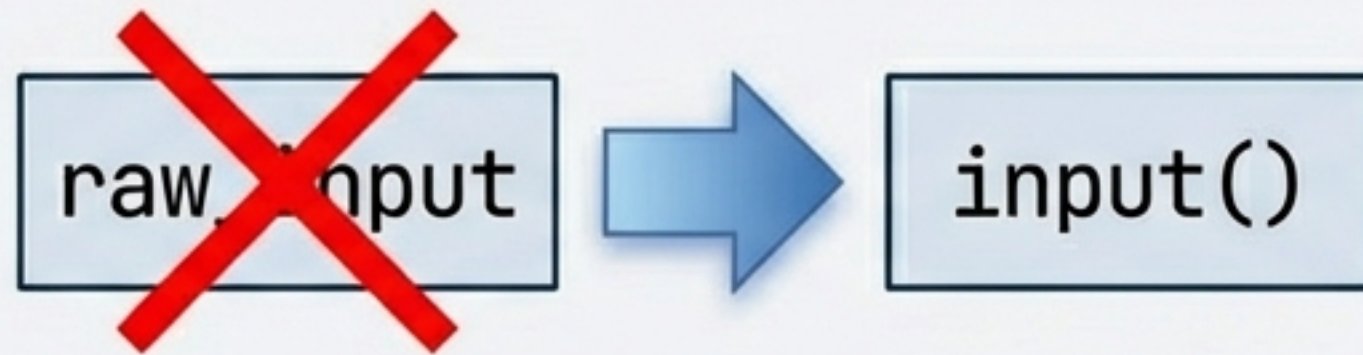
The image shows a screenshot of a code editor window titled "modernizado.py - Python 3.10+ - Modern IDE". The editor contains the following Python code:

```
print("Bienvenido")
# raw_input renombrado a input
nombre = input("Introduce tu nombre: ")
# Conversión explícita a entero para comparación segura
edad = int(input("Introduce tu edad: "))
if edad >= 18:
    print("Eres mayor de edad")
else:
    print("Eres menor de edad")
print("Fin del programa")
```

At the bottom right of the editor, there is a green button with a checkmark icon and the text "Compiled Successfully".

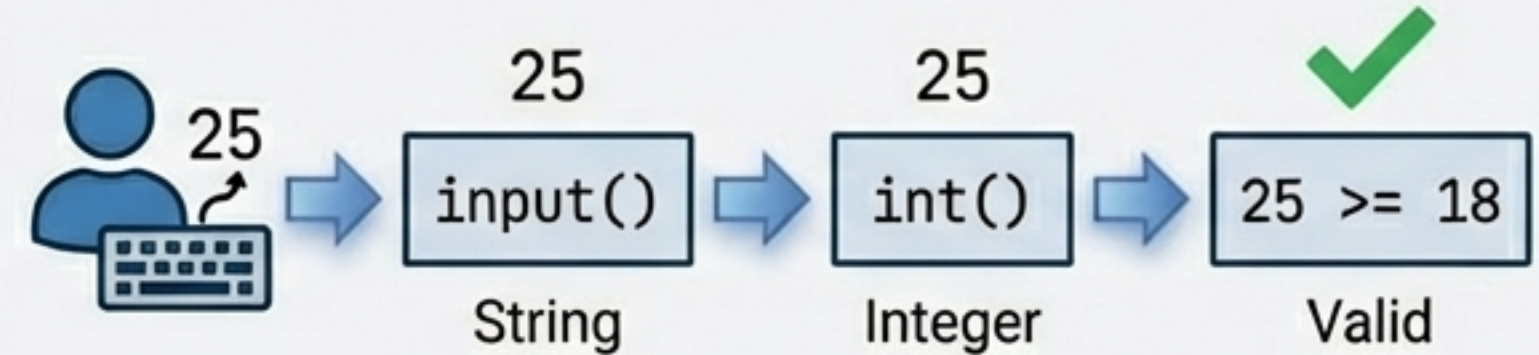
Justificación Técnica: Seguridad y Entradas

Eliminación de raw_input



Unificación en Python 3.
Siempre captura **string**.

Corrección de Tipado



En el código legado, **edad = input(...)** era un riesgo de ejecución de código arbitrario. La solución moderna usa **int()** para asegurar que operamos con números y prevenimos inyecciones.

Justificación Técnica: Sintaxis y Claridad

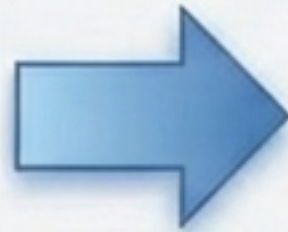
Insight 3: La Función Print



Python 2

Transforming

```
print "..."
```



```
print("...")
```

Transformar **print** en una función permite pasar argumentos adicionales (como **end=**, **sep=**) y sigue la filosofía de consistencia de Python 3.

Insight 4: Claridad del Código

Se mantiene la estructura **if/else** por ser legible y correcta, pero ahora opera sobre tipos de datos seguros y definidos, eliminando la ambigüedad de tipos.

Impacto Operativo: ¿Por qué es Vital la Versión?

Riesgos de ignorar el versionado en entornos profesionales.

Obsolescencia Tecnológica



Python 2 no recibe actualizaciones de seguridad desde 2020. Mantenerlo es una vulnerabilidad activa.



Deuda Técnica



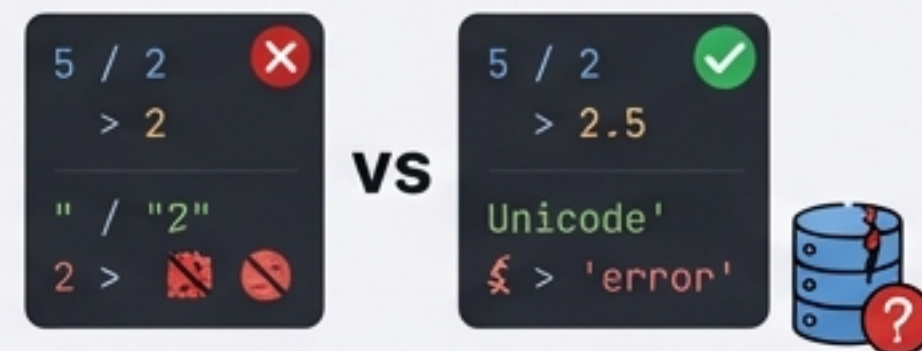
El coste de migrar **aumenta con el tiempo**. Las librerías modernas (NumPy, Pandas, Django) ya no soportan versiones antiguas.



Integridad de Datos



Diferencias sutiles (como la división / o el manejo de Unicode) pueden corromper datos silenciosamente sin lanzar errores inmediatos.



Resumen de Aprendizajes



Identificación: Distinguir código legacy (`print` sin paréntesis) de moderno (f-strings).



Comparativa: Entender los cambios profundos en I/O y matemáticas.



Diagnóstico: Verificar versiones y capacidades del entorno (`python --version`).



Refactorización: Transformar scripts inseguros en código robusto y tipado (Python 3.10+).

Especificaciones de Entrega



Entregable.pdf

- ✓ 1. Análisis completo de **fragmentos** (Parte 1).
- ✓ 2. **Tabla comparativa** explicada (Parte 2).
- ✓ 3. **Captura/Texto de comprobación** del entorno (Parte 3).
- ✓ 4. **Código modernizado** con **indentación correcta** (Parte 4).
- ✓ 5. Justificación técnica detallada y reflexión final.

Asegúrate de que el código esté presentado en bloques diferenciados y legibles. 